

FOODBRIDGE — DAY 8 NOTES

React Frontend Mentorship

Date: [Today's Date] | Topic: Register Page Connection & Toast Notifications

No new architectural features today — this session focused on wiring the Register form to the backend API, handling conditional fields, and setting up toast notifications for user feedback.

1. CONNECTING REGISTER TO BACKEND

The Register form needed to send user data to the backend API endpoint **POST /users**. Key steps:

- Build request object from form state variables
- Handle **organizationName** correctly (null for non-NGO roles)
- Make the API call using **axiosInstance**
- Handle success and error responses
- Show toast notifications for user feedback
- Redirect to login page on success

2. REQUEST OBJECT STRUCTURE

The backend expects this shape:

```
{
  fullName: string,
  email: string,
  password: string,
  phoneNumber: string,
  address: string,
  role: string (DONOR/NGO/VOLUNTEER),
  organizationName: string | null
}
```

IMPORTANT: organizationName must be null when role !== "NGO" because the backend expects null (not undefined or empty string).

```
// Correct approach using ternary operator:
organizationName: role === "NGO" ? organizationName : null
```

Why ternary?

- `role === "NGO"` → use the organizationName from state
- `role !== "NGO"` → send null (backend expects null)

3. COMPLETE HANDLEREGISTER FUNCTION

```
const handleRegister = (e) => {
  e.preventDefault();

  // 1. Build request object
  const requestData = {
    fullName,
    email,
    password,
    phoneNumber,
```

```

        address,
        role,
        organizationName: role === "NGO" ? organizationName : null
    };

    // 2. Make API call
    axiosInstance.post("/users", requestData)
        .then((response) => {
            console.log("Registration successful:", response.data);
            toast.success("Registration successful! Please login.");
            navigate("/login");
        })
        .catch((error) => {
            toast.error(error.response.data);
        });
}

```

4. IMPORTS NEEDED

```

// In Register.jsx:
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import axiosInstance from "../../api/axiosInstance";
import { toast } from "react-toastify";

// In App.jsx (for ToastContainer):
import { ToastContainer } from 'react-toastify';
import 'react-toastify/dist/ReactToastify.css';

```

5. TOAST NOTIFICATIONS EXPLAINED

react-toastify is a library that shows beautiful popup notifications.

ToastContainer (in App.jsx):

- Acts as the parent container for all toast notifications
- Must be rendered once at the top level
- Accepts props for positioning, timing, and appearance

toast functions (used in components):

- `toast.success("msg")` → Green notification for success
- `toast.error("msg")` → Red notification for errors
- `toast.warning("msg")` → Yellow notification for warnings
- `toast.info("msg")` → Blue notification for information

ToastContainer props used:

<code>position="top-right"</code>	→ Shows at top right corner
<code>autoClose={3000}</code>	→ Auto closes after 3 seconds
<code>hideProgressBar={false}</code>	→ Shows progress bar
<code>newestOnTop</code>	→ Newest toast appears on top
<code>closeOnClick</code>	→ Click to close
<code>pauseOnFocusLoss</code>	→ Pauses when tab loses focus
<code>draggable</code>	→ Can drag to dismiss
<code>pauseOnHover</code>	→ Pauses timer on hover
<code>theme="light"</code>	→ Light color theme

6. ERROR HANDLING WITH TOAST.ERROR()

Current implementation:

```
toast.error(error.response.data);
```

This works because the backend sends the error message directly in `error.response.data`.

Alternative approach (more robust):

```
toast.error(error.response?.data?.message || "Registration failed. Please try again.");
```

7. OPTIONAL CHAINING EXPLAINED (?)

Optional chaining (?) safely accesses nested object properties without throwing an error if a property doesn't exist.

```
// Without optional chaining (■ dangerous):
const message = error.response.data.message;
// If error.response or data is undefined → CRASHES!

// With optional chaining (■ safe):
const message = error.response?.data?.message;
// If error.response is undefined → returns undefined (no crash)
// If error.response.data is undefined → returns undefined (no crash)
```

Why use it in toast?

```
toast.error(error.response?.data?.message || "Registration failed");
```

```
// Shows backend message if available, otherwise shows fallback
```

8. THE LOGICAL OR (||) OPERATOR

The `||` operator returns the first *truthy* value.

Truthy values in JavaScript:

- Non-empty strings: "hello"
- Numbers: 42 (except 0)
- Objects: {}
- Arrays: []
- true

Falsy values in JavaScript:

- undefined
- null
- "" (empty string)
- 0
- false
- NaN

Example:

```
error.response?.data?.message || "Registration failed"  
// If left side has a value → use it  
// If left side is undefined/null/empty → use fallback
```

9. TOAST SETUP CHECKLIST

- Install react-toastify: `npm install react-toastify`
- Add ToastContainer in App.jsx with position, autoClose, etc.
- Add CSS import in App.jsx: `import 'react-toastify/dist/ReactToastify.css'`
- Add toast import in Register.jsx: `import { toast } from 'react-toastify'`
- Use `toast.success()` for success cases
- Use `toast.error()` for error cases

10. KEY LEARNING POINTS

1. Always set `organizationName` to `null` for non-NGO roles — backend expects `null`, not `undefined` or empty string
2. `ToastContainer` must be in `App.jsx` (once, at top level) — not in individual components
3. `toast.success()` → green for success | `toast.error()` → red for errors
4. `error.response.data` contains backend error messages — your backend sends the message directly in `.data`
5. Don't overcomplicate error handling if it works — simple code is better than overly complex code
6. State updates are async (remember Day 7 notes!) — not relevant here since we don't use `setUser` in `Register`

11. INTERVIEW-READY ANSWERS

Q: How do you handle form submission in React with API calls?

A: In React, form submission is handled by the `onSubmit` event on the form element. The handler calls `e.preventDefault()` to prevent the default browser reload. We gather data from component state (`useState`), build a request object, and make an API call using `axios`. On success, we show a toast notification and navigate to another page using `useNavigate()`. On error, we show an error toast with the backend's error message.

Q: How do you handle conditional fields in a form (like `organizationName` only for NGO)?

A: We conditionally render the field using `{role === "NGO" && (...)}` so the input only appears when the user selects "NGO". However, the data for that field exists in state regardless. When building the request object, we use a ternary operator to send `null` for non-NGO roles: `organizationName: role === "NGO" ? organizationName : null`. This ensures the backend receives the correct data shape.

Q: What is react-toastify and how do you set it up?

A: `react-toastify` is a library for displaying popup notifications in React apps. Setup involves: 1) installing the package, 2) adding `ToastContainer` to the root component (`App.jsx`) with configuration props, 3) importing the CSS file once, and 4) using `toast.success()` or `toast.error()` anywhere in components. The `ToastContainer` acts as a portal that renders notifications outside the normal component tree, so notifications work regardless of where they're triggered.

12. NEXT STEPS (DAY 9)

1. **Axios Interceptor** — Automatically attach JWT token to every request. Check localStorage for token before each request. Add Authorization: Bearer <token> header.
2. **Auto-logout on token expiry** — Decode JWT to get exp field. Check if token is expired before each request. Auto-logout and redirect to login page.
3. **Login redirect fix** — Add { replace: true } to navigate(). Prevents back button from returning to login.
4. **Start Donor Dashboard** — Create Donation form. List My Donations.

13. GIT COMMAND

```
git commit -m "feat: connect register to backend with toast notifications"
```

14. UPDATED RESUME PROMPT FOR DAY 9

PROJECT OVERVIEW:

FoodBridge is a food donation platform with 4 user roles: DONOR, NGO, VOLUNTEER, ADMIN. Backend: Spring Boot + JWT + MySQL (running on http://localhost:8080). Frontend: React + Vite + Tailwind v4 + React Router + Axios + Context API + react-toastify.

WHAT'S ALREADY BUILT AND WORKING:

- Login page – calls /users/login, decodes JWT, stores in AuthContext + localStorage, redirects to role-based dashboard
- Register page – connected to POST /users, organizationName null for non-NGO, toast notifications, redirects to /login on success
- AuthContext – persists user across refresh (checks localStorage on app load)
- ProtectedRoute – blocks unauthenticated/wrong role access
- Navbar – shows Login/Register or Dashboard/Logout based on user state
- All 4 dashboards exist and are protected
- Toast notifications working (ToastContainer in App.jsx)
- setUser() async bug pattern noted – always use decoded data directly

WHAT'S NEXT (in order):

1. Add Axios interceptor to automatically attach JWT (Authorization: Bearer <token>) to protected requests
2. Add auto-logout when token expires (using exp field from decoded JWT)
3. Implement navigate(path, { replace: true }) in Login's redirect
4. Start building Donor Dashboard features

RULES FOR THIS SESSION:

- Teach step by step, one thing at a time
- Never give too much info at once
- Wait for me to finish each step
- Explain Tailwind classes briefly (one line each)
- Make ME write the code — guide me, don't write everything
- Only correct exact mistakes, don't rewrite my whole code

- Use arrow functions for components
- I know basic React — explain only project-specific decisions

END OF DAY 8 NOTES